

Ring All-Reduce over RDMA

Eager versus Rendezvous

Ido Minha

Sapir Tobias

1 Required functionality

The process group is created by `connect_process_group`, used through `pg_all_reduce`, and destroyed by `pg_close`. Processes form a directed ring and implement:

- **Reduce-Scatter:** reduce corresponding blocks while circulating them; after $P - 1$ steps, each rank owns one reduced block.
- **All-Gather:** circulate the reduced blocks for another $P - 1$ steps until every rank holds the full result.
- **All-Reduce:** Reduce-Scatter followed by All-Gather.

For an input of N bytes per rank, each rank communicates approximately

$$2 \frac{P-1}{P} N$$

bytes. Uneven block sizes are supported. Blocks are split into pieces and up to four pieces are outstanding, allowing communication and reduction to overlap.

2 Implementation

The collective layer is independent of Verbs and calls an abstract transport interface: send to the next rank, receive from the previous rank, and wait/test a request. Tags encode collective sequence, phase, ring step, and pipeline piece.

The concrete transport uses reliable-connected queue pairs and TCP only for bootstrap/control metadata. Two data paths are implemented:

- **Eager:** the receiver posts a registered receive buffer and the sender issues `IBV_WR_SEND_WITH_IMM`.
- **Rendezvous:** the receiver sends its registered destination address and `rkey`; the sender issues `RDMA_WRITE_WITH_IMM`.

During All-Gather this writes directly into the final output location.

Memory remains registered until the matching completion is consumed. The implementation supports `INT32`, `FLOAT32`, and `FLOAT64`, with `SUM`, `MIN`, and `MAX`.

3 Experiment

Experiments ran on `mlx-stud-01-mlx-stud-04` using the active `mlx5_0` InfiniBand device. Each protocol was forced explicitly, so the same Ring All-Reduce workload was measured once with Eager and once with Rendezvous. The parameters were:

Processes	2 and 4
Input type/operation	<code>INT32/SUM</code>
Input size	256 B–4 MiB per rank, 16 sizes
Pipeline piece	64 KiB
Maximum in-flight pieces	4
Measurements	2 trials $\times$ 50 iterations per size/protocol
Warmup	10 iterations

Protocol order was reversed in the second trial. Connection setup was outside the timed region. Each iteration’s collective latency was defined as the *maximum* latency reported by its ranks, since the collective finishes only when its slowest rank finishes. Figures show the median over 100 collective samples. All measured runs passed result verification.

4 Results

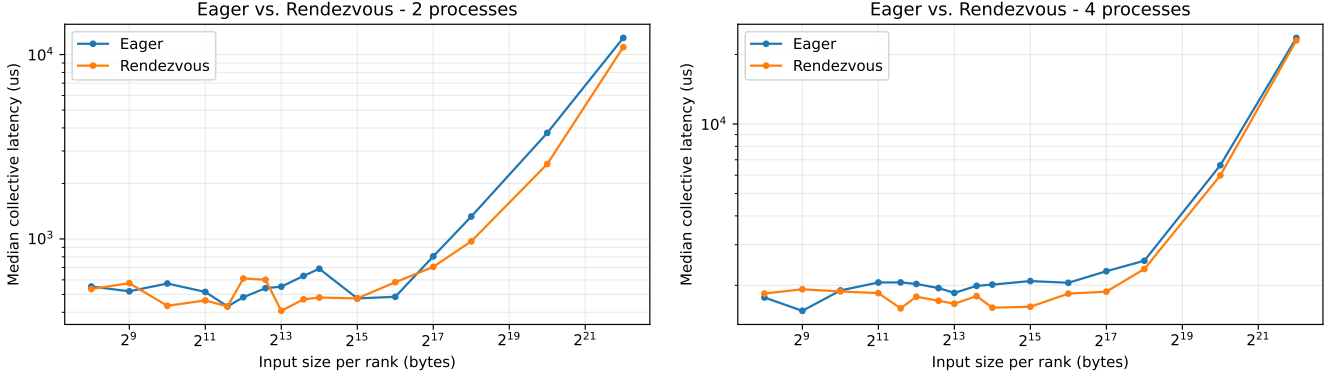


Figure 1: Median end-to-end All-Reduce latency. Both axes are logarithmic.

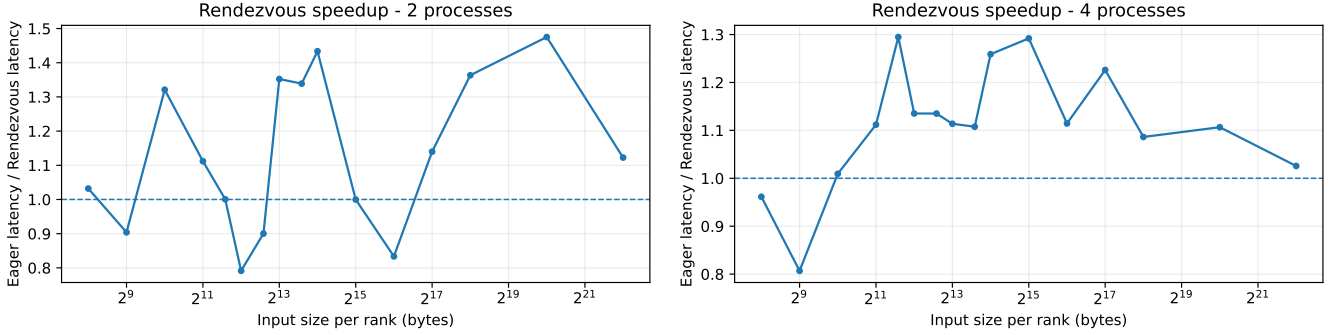


Figure 2: Ratio of Eager median latency to Rendezvous median latency. Values above one favor Rendezvous.

For two processes, Rendezvous wins 11 of 16 sizes, but the result is non-monotonic: Eager is faster at 4–6 KiB and 32–64 KiB, while Rendezvous is consistently better by median from 128 KiB upward. Its largest observed median speedup is $1.48\times$ at 1 MiB.

For four processes, Rendezvous wins 14 of 16 sizes and every tested size from 1 KiB upward. Its largest speedup is $1.29\times$ at 3 KiB. The extra ring steps amplify the benefit of direct writes and make Rendezvous preferable much earlier.

The original 4 KiB automatic cutoff is therefore only a heuristic. At exactly 4 KiB, Eager is $1.26\times$ faster with two processes, whereas Rendezvous is $1.14\times$ faster with four. A single process-independent cutoff is not supported by these measurements. A conservative policy for this implementation is to tune by group size: keep Eager only for the smallest four-process messages, while using Rendezvous for clearly large two-process messages. There is clearly some noise that we were unable to mitigate in this implementation, though we believe the cutoff to be relatively valid. This also shows that the costs shared between eager and rendezvous are the more dominant ones, rather than the differences in implementation.

5 Limitations and conclusion

The comparison measures the implemented protocols, including per-piece TCP control exchange and per-piece memory registration. It is therefore not a comparison against an ideal Eager implementation with a permanent pre-registered buffer pool. Within this implementation, however, Rendezvous scales better as the number of ring steps and payload size grow. The project satisfies the required ring collectives, pipelining, Verbs communication, direct large-message All-Gather path, and two-/four-process testing.

P	Size	Eager (μs)	Rendezvous (μs)	E/R	Winner
2	256 B	552.1	535.0	1.0	R
2	1 KiB	573.0	433.7	1.3	R
2	4 KiB	482.8	610.0	0.8	E
2	16 KiB	689.3	480.9	1.4	R
2	64 KiB	485.5	582.4	0.8	E
2	256 KiB	1322.9	970.3	1.4	R
2	1 MiB	3759.2	2548.4	1.5	R
2	4 MiB	12 312.7	10 968.9	1.1	R
4	256 B	1772.2	1843.2	1.0	E
4	1 KiB	1899.9	1882.2	1.0	R
4	4 KiB	2028.1	1786.5	1.1	R
4	16 KiB	2015.0	1600.5	1.3	R
4	64 KiB	2052.0	1841.4	1.1	R
4	256 KiB	2556.8	2353.9	1.1	R
4	1 MiB	6627.9	5989.4	1.1	R
4	4 MiB	23 603.8	23 013.0	1.0	R

Table 1: Selected median latencies. E/R above one favors Rendezvous.